CardioIA — Documento de Arquitetura

FIAP • Fase 6 | Sistema multiagente para análise de risco sintético

1. Visão geral

A solução combina um classificador de Machine Learning treinado com dados sintéticos e um fluxo de agentes que coordena validação, predição, seleção de regras e consolidação da resposta. A aplicação é executada pela linha de comando em sistema_multiagente_risco/src/main.py e utiliza Google ADK com Gemini para a coordenação dos agentes.

2. Componentes e responsabilidades

Componente	Responsabilidade	Implementação
Orquestrador	Inicia o fluxo e encaminha a tarefa ao agente responsável por analisar o caso.	src/agents_config.py
Analista de Risco	Valida os campos de entrada, registra o caso e chama a predição.	src/tools.py + modelo Joblib
Especialista em Protocolos	Consulta regras de resposta sintéticas de acordo com a classificação.	data/protocolos.json
Planejador de Resposta	Consolida os dados validados em uma saída estruturada.	Schema Pydantic RespostaFinal
Contexto e sessão	Compartilha estado e mantém histórico local da execução.	src/context.py + sessão SQLite
Modelo de ML	Produz classe e probabilidade a partir dos atributos numéricos.	models/modelo_risco_cardiaco.joblib

3. Fluxo, handoffs e tools

Entrada JSON → validação Pydantic → Orquestrador → Analista de Risco → modelo salvo → Especialista em Protocolos → Planejador de Resposta → JSON final. Os handoffs transferem o controle entre agentes especializados. As tools executam tarefas determinísticas, como validação/registro do caso, inferência do modelo e consulta ao arquivo de protocolos. Pydantic define os contratos de entrada e saída; o contexto compartilhado mantém os dados do caso ao longo do fluxo.

4. Exemplo ilustrativo de entrada e saída

Entrada simulada (JSON)	Saída ilustrativa (JSON resumido)

{
"idade": 58,
"frequencia_cardiaca": 112,
"spo2": 93,
"pressao_sistolica": 148,
"...": "demais campos exigidos"
}
	{
	"classificacao": "ALTO",
	"probabilidade": 0.9867,
	"protocolos": ["regras sintéticas aplicáveis"],
	"aviso": "uso acadêmico"
	}

O exemplo é ilustrativo e parcial: a aplicação exige todos os campos definidos pelo schema. A probabilidade indicada corresponde a uma simulação de teste já observada no projeto; a saída exata depende da entrada e dos artefatos usados.

5. Organização, configuração e execução

Na raiz estão o notebook de treinamento, artefatos_modelo/, docs/ e sistema_multiagente_risco/. Dentro do subsistema, src/ contém agentes, contexto, schemas e ferramentas; models/ contém o modelo e as colunas esperadas; data/ contém casos e protocolos; tests/ contém testes automatizados. Use Python 3.11 e instale as dependências declaradas em requirements.txt. Configure GOOGLE_API_KEY e GOOGLE_GENAI_USE_VERTEXAI=FALSE no arquivo local .env, sem versionar a chave. Execute a partir da raiz: py -3.11 -m sistema_multiagente_risco.src.main.

6. Limites e segurança

Dados, rótulos e protocolos são sintéticos; métricas elevadas não validam o sistema no mundo real. O protótipo não deve orientar condutas assistenciais ou decisões médicas. Qualquer aplicação real exigiria dados autorizados, validação independente, avaliação de vieses e segurança, governança, monitoramento e supervisão especializada.

Uso acadêmico exclusivamente. Não utilizar para diagnóstico, triagem, prescrição ou decisões médicas reais.